

Registro para Minutas 2

Estudiante	Minuta 2			
	Tareas realizadas	Resultados obtenidos	Próximo paso	Fecha / Evaluación
Rayan Bentaleb Zaki	- Refactorización de la clase GrafoUrbano en modelos.py: Migración de la estructura "diccionario de diccionarios" a una lista de adyacencia (diccionario de listas de tuplas) para optimizar la iteración de vecinos en el futuro algoritmo de Backtracking. - Actualización de la clase Pedido: Implementación de lógica de generación aleatoria en el constructor para permitir la creación de instancias proporcionando únicamente el identificador. - Desarrollo de la función GenerarGrafos en utils/generador_escenarios.py: Creación de un motor de generación de escenarios masivos que garantiza la conectividad del grafo mediante un camino base y añade complejidad mediante aristas aleatorias.	- Infraestructura de datos ajustada a los requisitos técnicos de la práctica, mejorando la eficiencia para la búsqueda de rutas. - Capacidad de generar escenarios de estrés (50 nodos y 50 pedidos) de forma automática y en segundos, facilitando las pruebas de defensa del proyecto. - Integración exitosa de la mejora de recursividad (Tema 1) dentro del flujo de generación masiva del grafo y el gestor de pedidos.	- Iniciar el diseño de las funciones de poda y estimación de cota para el algoritmo de ruteo por Backtracking (Tema 5). - Establecer la estructura para la toma de métricas de tiempos de ejecución y nodos explorados	27/04/2026 / Pendiente
Óscar López Rojo	- Investigación y planteamiento de la ecuación de recurrencia y casos base para la mochila 0/1 adaptada a python para incluirla en el algoritmo de selección. - Desarrollo de código para el Módulo de Selección (dp_seleccion.py).	- Algoritmo Bottom-Up funcional que garantiza la solución óptima de beneficio. - Resolución de subproblemas implementada	- Redactar el análisis de complejidad teórica ($O(n * C)$) en el Informe Técnico	27/04/2026/ Pendiente

	Comentarios incluidos para facilitar la lectura. Se implementa un algoritmo Bottom-Up, empezando desde un caso base minúsculo hasta los casos más complejos.	eficientemente en matriz bidimensional. - Código preparado para integración sin errores de índices. Para ello contemplamos el caso base y utilizamos un índice ($i - 1$) en los bucles.	- Programar los test unitarios (test_dp.py) para comprobar si el algoritmo contiene errores a la hora de juntar todo el código. Implementar y juntar los algoritmos en el main.py.	
Jorge Sáez Villate	- Implementación acerca de los generadores de escenarios básicos y capacidad crítica. - En la clase main.py implementación acerca de la generación recursiva de un número n de pedidos y la creación del grafo para insertar los diferentes pedidos. - Conflictos acerca de las rutas a la hora de ejecutar el archivo.	- Integración correcta de pedidos almacenados en cada escenario correspondiente donde se han creado. - Capacidad de generar y poder cargar gran cantidad de nodos y pedidos. Desarrollo del main.py para la creación de pedidos e integrarlos en el grafo urbano - Una clara organización acerca de las rutas, mediante	- Realizar un análisis de comparación entre la programación dinámica y los algoritmos voraces. - Verificar que la selección de pedidos realizada por la DP coincide con la solución óptima teórica en los escenarios básicos.	27/04/2026/Pendiente

		el comando -m y así poder obtener todos los archivos de los cuales tiene funciones y ejecutar correctamente	- Mostrar grafo con los pedidos en cada nodo	
--	--	---	--	--

Nota: Cada minuta se evalúa en base a 10 y luego se promedia su nota. Todos los estudiantes tienen que presentar tres avances.

```
Cargando 5 pedidos en escenario_basico.json
Pedidos guardados de forma en correcta en escenario_basico.json
Cargando 50 pedidos en escenario_capacidad.json
Pedidos guardados de forma en correcta en escenario_capacidad.json
```

Pedidos almacenados en los escenarios

```
=== INICIANDO SISTEMA UAH-ROUTE ===
--- 1. Cargando pedidos desde escenario de capacidad critica ---
--- 2. Construcción de la Ciudad ---
Generando escenario con 50 nodos y 50 pedidos...
Grafo construido con 50 nodos.

--- RESULTADO DE LA PROGRAMACIÓN DINÁMICA ---
Beneficio máximo: 1179€
Peso total cargado: 100kg
Pedidos seleccionados para el camión: ['P4', 'P7', 'P13', 'P15', 'P18', 'P21', 'P22', 'P26', 'P27', 'P31', 'P33', 'P37', 'P38', 'P45', 'P46', 'P47', 'P48', 'P49', 'P50']
```

Prueba de generar grafo y resultado acerca de la programación dinámica